

NATIONAL RESEARCH UNIVERSITY HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "HSE and University of London Double Degree Programme in Data
Science and Business Analytics"

Programming project report

on the topic Numerical Algorithms of Linear Algebra

Student:

group БПА/Д232

Sign

З. Ю. Зинкин

Name, Patronymic, Surname

4.02.2025

Date

Supervisor:

Дмитрий Витальевич Трушин

Name, Patronymic, Surname

доцент / ФКН НИУ ВШЭ

Position / Company

4.02.2025

Date

Grade

Sign

Moscow 2025

Contents

1	Introduction	3
2	Description of functional and non-functional requirements for the program project	4
2.1	Functional requirements	4
2.2	Non-functional requirements	4
3	Theoretical part	5
3.1	Main definitions	5
3.2	Algorithms	6
3.2.1	Householder reflection	6
3.2.2	Givens rotations	8
3.2.3	QR decomposition	8
4	Library LinAlgTools	10
5	Testing	10
6	Conclusion	10

Abstract

The main focus of the programming project is to study matrix decomposition algorithms from a theoretical point of view with its subsequent implementation on the C++ language.

1 Introduction

Linear Algebra provides extremely useful tools for computer science and other engineering fields. However, even the simplest algorithms of linear algebra cannot be directly used in practice due to inefficiencies. Consider the matrix multiplication algorithm: in its naive implementation the complexity is $O(n^3)$, so in order to multiply two 1000x1000 matrices, you would need a billion operations!

In order to create faster algorithms a field of Numerical Linear Algebra has been developed recently. As one of its achievements I can mention Coppersmith Winograd matrix multiplication algorithm with complexity $O(n^{2.376})$ ¹, which reduced the previous number of operations to just 13mln.

The goal of my programming project is to learn modern algorithms of Numerical Linear Algebra with a focus on matrix decomposition algorithms, which represent a matrix in a certain way for further computation. In order to have a profound understanding of each of the algorithms, it is necessary to implement a C++ library from scratch.

The library itself contains a class for matrix representation with implementation of matrix arithmetic and other necessary methods. Decomposition algorithms are implemented based on the aforementioned matrix class. The whole library will be tested using Googletest[2].

The information for the theoretical part of the project is taken from the books, which are cited at the end of the paper. Citation of exact chapters is used throughout the theoretical part when needed.

The result of the project is a C++ library LinAlgTools[3] with the following functionality:

- QR Decomposition
- SVD Decomposition

Theoretical part can help you understand the underlying theory of each of the algorithms. The technical part of the project is described in detail in the Library LinAlgTools section of the paper. The results of the testing using Googletest[2] are documented in the Testing section.

¹In general the algorithm is not widely used due to its limitations on matrix conditions

2 Description of functional and non-functional requirements for the program project

2.1 Functional requirements

- A *Matrix* class, which allows users to perform matrix arithmetic. The class must be extendable for potential requirements of to-be implemented algorithms.
- A *SubMatrix* and *ConstSubMatrix* classes for handling particular part of an existing matrix without copying data.
- Implementation of numerically stable algorithms:
 - QR Decomposition using Householder rotations - representation of a matrix as a product of Q and R matrices, which are computed using Householder rotations.
 - QR Decomposition using Given's rotations - representation of a matrix using specific Q and R matrices, which are computed using Givens rotations.
 - SVD Decomposition - representation of a matrix as a product of U , Σ and V^* matrices, where U , V are unitary and Σ is a diagonal matrix.
- Extensive testing must be done in order to guarantee proper functioning of the library.
- And other requirements which might be relevant during the development of the library.

2.2 Non-functional requirements

- *GCC* compiler with C++23 standart support.
- Code formatter *clang-format* to automatically format code based on the Microsoft style with additional changes.
- *Git*, a version control system, and a remote repository on *GitHub* [3].
- A third-party library *Googletest* [2] for writing tests.
- Minimum RAM requirement: 4GB.
- And other requirements which might be relevant during the development of the library.

3 Theoretical part

3.1 Main definitions

Definition 1. An $m \times n$ matrix over a field $\mathbb{F}$ (either $\mathbb{C}$ or $\mathbb{R}$) is a rectangular array of elements of $\mathbb{F}$, arranged in m rows and n columns:

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

A set of all $m \times n$ matrices over a field $\mathbb{F}$ we denote $\mathbb{F}^{m \times n}$.

Definition 2. A matrix $A \in \mathbb{F}^{m \times n}$ is called *diagonal* if all its elements not on the diagonal are 0:

$$\begin{bmatrix} a_{11} & 0 & 0 & \dots & 0 & \dots & 0 \\ 0 & a_{22} & 0 & \dots & 0 & \dots & 0 \\ 0 & 0 & a_{33} & \dots & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{mk} & \dots & 0 \end{bmatrix}$$

A special case of a diagonal matrix is an $n \times n$ *Identity* matrix:

$$I_n = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{bmatrix}$$

Definition 3. A matrix $A \in \mathbb{F}^{m \times n}$ is called *upper triangular* if all its elements below the diagonal are 0:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1k} & \dots & a_{1n} \\ 0 & a_{22} & a_{23} & \dots & a_{2k} & \dots & a_{2n} \\ 0 & 0 & a_{33} & \dots & a_{3k} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & a_{mk} & \dots & a_{mn} \end{bmatrix}$$

Definition 4. A vector is an $1 \times n$ or $n \times 1$ matrix:

$$\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \quad \text{or} \quad [a_1 \quad a_2 \quad \dots \quad a_n]$$

A set of all $m \times 1$ or $1 \times m$ vectors over $\mathbb{F}$ we denote $\mathbb{F}^m$.

Definition 5. A *Hermitian conjugate* of a matrix $A \in \mathbb{C}^{m \times n}$ is an $n \times m$ matrix, obtained by transposing A and applying complex conjugate to each of entry: $(A^*)_{ij} = \overline{A_{ji}}$.

Remark. For real matrices *Hermitian conjugate* coincides with transposition of the matrix: $A^* = A^T$.

Definition 6. A matrix $A \in \mathbb{C}^{m \times n}$ is called *Hermitian* if $A^* = A$.

Definition 7. A matrix $A \in \mathbb{C}^{m \times n}$ is called *Unitary* if $A^* = A^{-1}$. As a consequence, $A^*A = AA^* = I$.

Definition 8. A matrix $A \in \mathbb{R}^{m \times n}$ is called *Orthogonal* if $A^T = A^{-1}$. As a consequence, $A^TA = AA^T = I$.

Statement 1. A product of unitary (orthogonal) matrices is also unitary (orthogonal).

Definition 9. A *scalar product* is a bilinear form $\langle \cdot, \cdot \rangle : \mathbb{F}^n \times \mathbb{F}^m \rightarrow \mathbb{F}$. In a Euclidean space, such as $\mathbb{R}^m$ or $\mathbb{C}^m$ the scalar product of two vector $v, w \in \mathbb{C}^m$ is defined in the following way:

$$\langle v, w \rangle = w^* v = \sum_{i=1}^m v_i \overline{w_i}$$

Definition 10. A 2-norm, or Euclidean norm, on an Euclidean space is a function: $\|\cdot\|_2 : \mathbb{F}^n \rightarrow \mathbb{R}_+$.
Let $v \in \mathbb{F}^n$:

$$\|v\|_2 = \sqrt{\langle v, v \rangle} = \sqrt{\sum_{i=1}^m |v_i|^2}$$

Geometrically, a norm returns the distance from the origin to the point in the space.

3.2 Algorithms

3.2.1 Householder reflection

A *Householder reflection* is a linear transformation which reflects a vector about a hyperplane.[1]

Given a vector $v \in \mathbb{F}^m$, an $m \times m$ matrix P (or more commonly $H(v)$) of the form

$$P = H(v) = I - \frac{2}{\|v\|_2} v v^*$$

is called a *Householder reflector* or *Householder matrix*. The matrix allows us to reflect vector along the hyperplane, spanned by v . Moreover, a Householder matrix is unitary and hermitian ($H^{-1} = H^* = H$), which turns out to be key in the *QR* decomposition algorithm.

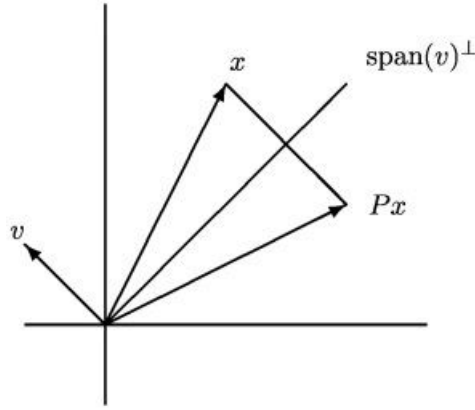


Figure 1: Householder reflection applied to a vector x

Statement 2. $\forall a, b \in \mathbb{C}^n : \|a\|_2 = \|b\|_2, \exists \gamma \in \mathbb{C} : |\gamma| = 1$ and $v \in \mathbb{C}^n : \|v\|_2 = 1$, such that $H(v)a = \gamma b$.

Proof. Using direct computations we can find the required γ and v :

$$H(v) = I - 2vv^*$$

$$H(v)a = a - 2vv^*a = \gamma b$$

$$2(v^*a)v = a - \gamma b \tag{1}$$

From here it follows that v must be of the form:

$$v = \mu \frac{a - \gamma b}{\|a - \gamma b\|_2}$$

But since we are proving the existence, we can set μ to 1, hence:

$$v = \frac{a - \gamma b}{\|a - \gamma b\|_2}$$

Plugging it into (1):

$$\begin{aligned} 2 \frac{(a - \gamma b)^*}{\|a - \gamma b\|_2} a \frac{a - \gamma b}{\|a - \gamma b\|_2} &= a - \gamma b \\ 2 \frac{(a - \gamma b)^*}{\|a - \gamma b\|_2} a \frac{1}{\|a - \gamma b\|_2} &= 1 \\ 2(a - \gamma b)^* a &= \|a - \gamma b\|_2^2 \\ 2(a - \gamma b)^* a &= (a - \gamma b)^* (a - \gamma b) \\ 2a^* a - 2\bar{\gamma} b^* a &= a^* a + b^* b - \gamma a^* b - \bar{\gamma} b^* a \\ \bar{\gamma} b^* a &= \text{Re}(\bar{\gamma} b^* a) \\ \gamma &= \pm \frac{b^* a}{|b^* a|} \end{aligned}$$

If $a \perp b$, γ can be any number, such that $|\gamma| = 1$. If $a \parallel b$, then we should carefully choose the sign to guarantee numeric stability of the transformation. $\square$

Corollary 1. $\forall x \in \mathbb{C}^n \exists v \in \mathbb{C}^n : \|v\|_2 = 1$ and

$$H(v)x = \gamma \begin{bmatrix} \|x\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$v = \frac{x - \gamma \|x\|_2 e_1}{\|x - \gamma \|x\|_2 e_1\|_2}, \quad \gamma = -\frac{x_1}{|x_1|}$$

The corollary yields an algorithm for transforming a vector:

Algorithm 1 Householder Vector Reduction

```

1: function HOUSEHOLDERREDUCTION( $x \in \mathbb{C}^n$ )
2:    $v = x$ 
3:    $v_1 = x_1 - \text{sign}(x_1) \cdot \|x\|_2$ 
4:    $v = \frac{v}{\|v\|}$ 
5:   return  $v$ 
6: end function
```

And if we want to apply the transformation to a matrix from the left (right), we can use:

Algorithm 2 Householder Left Reflection

```

1: function HOUSEHOLDERLEFTREFLECTION( $A \in \mathbb{C}^{m \times n}, v \in \mathbb{C}^m$ )
2:    $M = A - (2v) \cdot (v^* A)$ 
3:   return  $M$ 
4: end function
```

Algorithm 3 Householder Right Reflection

```

1: function HOUSEHOLDERRIGHTREFLECTION( $A \in \mathbb{C}^{m \times n}$ ,  $v^* \in \mathbb{C}^n$ )
2:    $M = A - (Av^*) \cdot (2v)$ 
3:   return  $M$ 
4: end function

```

3.2.2 Givens rotations

In cases when we need to zero specific elements of the matrix, *Givens rotations* are our tool of choice. A *Givens rotation* is a linear transformation which amounts to a counterclockwise rotation by an angle of θ . The matrix of a *Givens rotation* is of the form:

$$G(i, j, \theta) = \begin{bmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \cdots & \cos \theta & \cdots & -\sin \theta & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & \sin \theta & \cdots & \cos \theta & \cdots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{bmatrix} \begin{matrix} \\ \\ i \\ \\ j \\ \\ \end{matrix}$$

Notice that G is almost the identity matrix. We can exploit this structure to achieve faster computation: instead of having a matrix A by G to apply the transformation, we can directly calculate the rows which are affected by G .

3.2.3 QR decomposition

Statement. For any matrix $A \in \mathbb{F}^{m \times n}$ exist a unitary matrix $Q \in \mathbb{F}^{m \times m}$ and an upper-triangular matrix $R \in \mathbb{F}^{m \times n}$, such that $A = QR$.

Proof. Using Householder rotations (for convenience $A \in \mathbb{F}^{5 \times 3}$):

$$A = \begin{bmatrix} \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \\ \star & \star & \star \end{bmatrix} \xrightarrow{H(v_1)} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \\ 0 & \star & \star \end{bmatrix} \xrightarrow{H(v_2)} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \\ 0 & 0 & \star \\ 0 & 0 & \star \end{bmatrix} \xrightarrow{H(v_3)} \begin{bmatrix} \star & \star & \star \\ 0 & \star & \star \\ 0 & 0 & \star \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = R$$

$$H_3 H_2 H_1 A = R$$

$$A = H_1^{-1} H_2^{-1} H_3^{-1} R = H_1 H_2 H_3 R = QR$$

□

The proof yields an algorithm:

Algorithm 4 Householder QR Decomposition

```

1: function HOUSEHOLDERQR( $A \in \mathbb{F}^{m \times n}$ )
2:    $Q = I_m$ ,  $R = A$ 
3:   for  $k = 1$  to  $n$  do
4:      $\mathbf{x} = R[k : m, k]$ 
5:      $v = \text{HouseholderReduction}(\mathbf{x})$ 
6:      $R[k : m, k : n] = \text{HouseholderLeftReflection}(R[k : m, k : n], v)$ 
7:      $Q[k : m, :] = \text{HouseholderLeftReflection}(Q[k : m, :], v)$ 
8:   end for
9:    $Q = Q^T$ 
10:  return  $Q, R$ 
11: end function

```

Proof. Using Householder rotations (for convenience $A \in \mathbb{F}^{3 \times 2}$):

$$A = \begin{bmatrix} \star & \star \\ \star & \star \\ \star & \star \end{bmatrix} \xrightarrow{G_{13}} \begin{bmatrix} \star & \star \\ \star & \star \\ 0 & \star \end{bmatrix} \xrightarrow{G_{12}} \begin{bmatrix} \star & \star \\ 0 & \star \\ 0 & \star \end{bmatrix} \xrightarrow{G_{23}} \begin{bmatrix} \star & \star \\ 0 & \star \\ 0 & 0 \end{bmatrix} = R$$

$$G_{23}G_{12}G_{13}A = R$$

$$A = G_{13}^*G_{12}^*G_{23}^*R = QR$$

□

The proof once again yields an algorithm:

4 Library LinAlgTools

This section of the paper will contain description of the library's architecture, code references and examples of library's usage.

5 Testing

In order to ensure the proper functioning of the library, Googletest [2] is used. This section of the paper will contain results of the tests with additional performance graphs.

6 Conclusion

This section of the paper will contain the results of the development of the programming project with a brief overview of the accomplished work.

References

- [1] Charles F. Van Loan Gene H. Golub. Matrix computations, 4th edition. URL: <https://disk.yandex.ru/i/11ZWE2ud-IZtMA>, 2013.
- [2] Google. Googletest. URL: <https://github.com/google/googletest>, 1.15.2.
- [3] Zinkin Zakhar. LinAlgTools. URL: <https://github.com/Avgustineiw/LinAlgTools>.